How the Browser Builds the DOM

Imagine ordering furniture from IKEA.

HTML = flat instruction sheet.
DOM = fully assembled furniture with screws, structure, relationships.

JavaScript **cannot work with HTML text**.
It only works with the constructed **DOM Tree**.

Definition:

DOM is the structured representation of an HTML document created by the browser, where every element becomes an object that JavaScript can manipulate to change the content, structure, or style of the page without reloading

Key Points Interviewers Expect

DOM is **created by the browser**, not JavaScript.

It converts HTML into a **tree of nodes** (Element, Text, Attribute).

JavaScript uses DOM APIs like:

- getElementById
- querySelector
- createElement

Enables **dynamic updates without page reload**.

DOM is **language-independent**, but mostly used with JavaScript.

```
<h1 id="title">Hello</h1>
```

Browser converts it into DOM:

```
Document
 └── html
     └── body
         └── h1 (object)
```

JavaScript can now do:

```
document.getElementById("title").textContent = "Hello Prajwal";
```

This changes the page **without refreshing** because we modified the DOM.

Q: Why is DOM slow?
Because every change may trigger:

Recalculate Style → Layout → Paint → Composite

Selecting Elements Like a Surgeon

Avoid outdated:

```
getElementById
```

Modern scalable selection:

```
querySelector()
querySelectorAll()
```

Why?
Because CSS selectors scale with complexity.

✅ **1. Always Start With the Narrowest Scope**

❌ Rookie mistake (search entire document):

```
const buttons = document.querySelectorAll(".btn");
```

✅ Surgical approach (limit search area):

```
const container = document.getElementById("checkout");
const buttons = container.querySelectorAll(".btn");
```

Why this matters

Browsers must walk the DOM tree to resolve selectors.

Scoped queries are dramatically faster in large apps.

Interview Line:
"I reduce selector search space to avoid global DOM traversal."

✅ **4. Cache What You Select (DOM Queries Are Not Free)**

❌ Re-querying every time:

```
function update() {
  const el = document.querySelector(".price");
  el.textContent = calculate();
}
```

✅ Cache once:

```
const priceEl = document.querySelector(".price");

function update() {
  priceEl.textContent = calculate();
}
```

This prevents repeated DOM resolution work.

Understand Live vs Static Collections (Common Interview Trap)

getElementsByClassName → LIVE collection

```
const items = document.getElementsByClassName("item");
```

Updates automatically when DOM changes.

querySelectorAll → STATIC snapshot

```
const items = document.querySelectorAll(".item");
```

Does NOT update.

Q: Why avoid live collections?
They cause unpredictable performance bugs.

Q: Fastest selector?
ID selectors — but rarely scalable.

✅ **What You Can Put Inside (Common Cases)**

1️⃣ **Select by ID (#)**

```
document.querySelector("#header");
```

Same as CSS:

#header { }

2️⃣ **Select by Class (.)**

```
document.querySelector(".card");
```

3️⃣ **Select by Tag Name**

```
document.querySelector("button");
```

4️⃣ Select Nested Elements (Very Common)

```
document.querySelector(".card button");
```

Means:

Find a button **inside** .card.

5️⃣ Select Direct Child (>)

```
document.querySelector(".list > li");
```

Only direct children.

6️⃣ Select Using Attribute

```
document.querySelector('[data-id="101"]');
```

Super useful in real apps:

```
<button data-action="save"></button>
```

```
document.querySelector('[data-action="save"]');
```

7️⃣ Combine Selectors (More Precise)

```
document.querySelector("input[type='email']");
```

8️⃣ Multiple Conditions

```
document.querySelector(".card.active");
```

Element that has **both classes**.

✅ If You Want ALL Matches → use querySelectorAll

```
document.querySelectorAll(".card");
```

Returns a **NodeList** (like array).

getElementById(id)

```
document.getElementById("idName");
```

Changing the DOM Structure

1️⃣ Create a New Element

```
const li = document.createElement("li");
li.textContent = "New Item";
```

This only creates it **in memory** (not visible yet).

2️⃣ Insert Into the DOM

```
const list = document.getElementById("todo");
list.appendChild(li); // adds at the end
```

```
list.prepend(li);          // insert at start
list.before(li);          // insert before element
list.after(li);           // insert after element
```

4️⃣ Remove an Element

```
li.remove();
```

Or:

```
list.removeChild(li);
```

5️⃣ Replace an Element

```
const newNode = document.createElement("p");
newNode.textContent = "Updated";

oldNode.replaceWith(newNode);
```

8️⃣ Text vs HTML Insertion (Security Matters)

```
el.textContent = "<b>Hello</b>";   // safe, treated as text
el.innerHTML   = "<b>Hello</b>";   // parsed as HTML
```

Q: Difference between append and appendChild?

"appendChild() inserts a single DOM node, while append() is a modern, more flexible method that can insert multiple nodes or text in one call."

Q. **Why is innerHTML dangerous?**

"innerHTML is dangerous because it parses raw HTML, which can execute malicious scripts (XSS), and it forces the browser to destroy and rebuild the DOM subtree, causing expensive reflows and loss of existing state. Modern code prefers DOM APIs or safe text insertion to make incremental updates."

Event System Deep Dive

The DOM Event System is how the browser turns **user actions** (click, scroll, input, keypress) into **JavaScript execution**.

Every modern UI — including React — ultimately relies on this native event pipeline

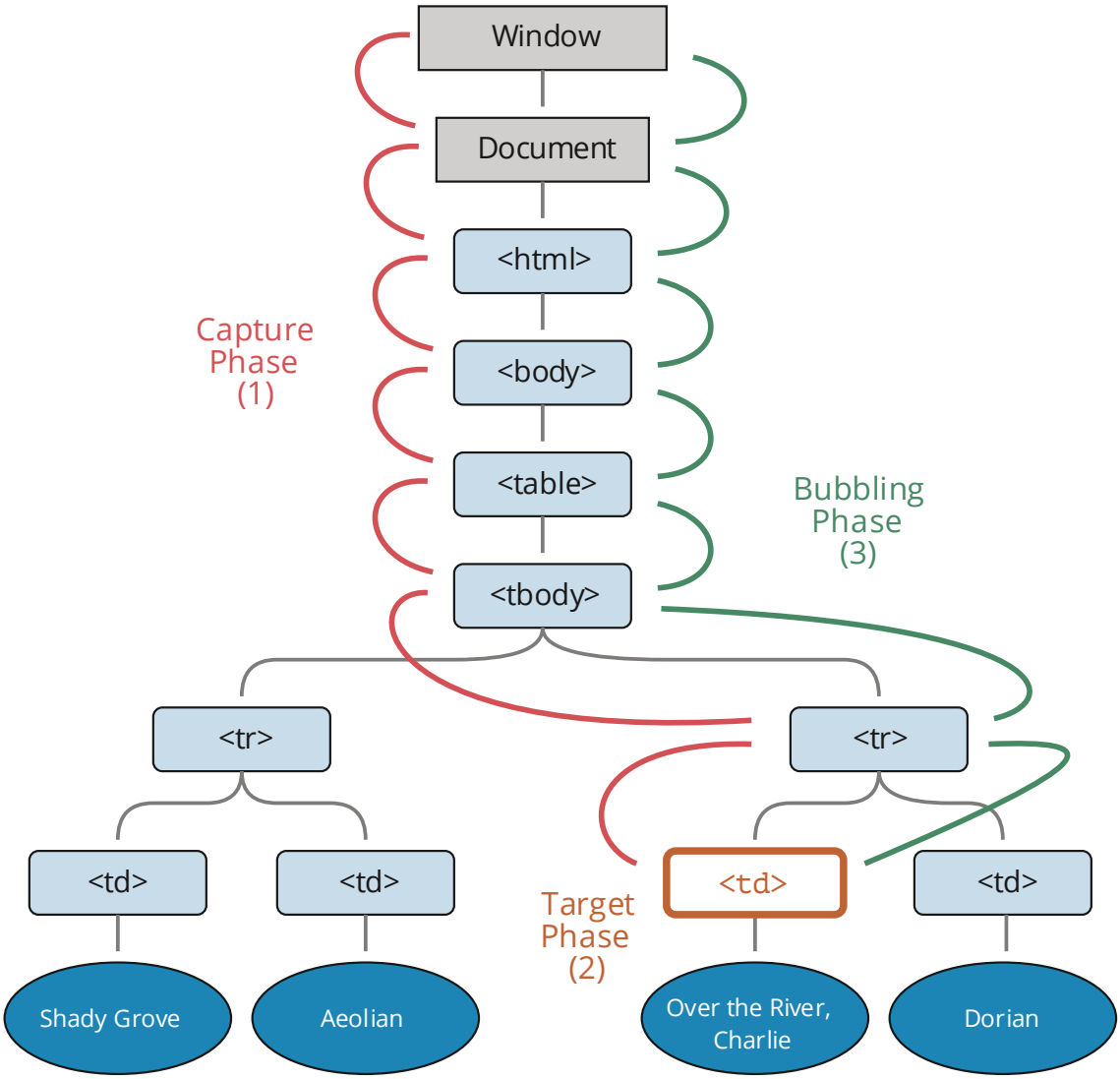
1. **Event Flow: The 3 Phases (Core Concept)**

   When you click a button, the event **does NOT go directly to that button**. It travels through the DOM in a defined path called **Event Propagation**.

   The Three Phases:

   1️⃣ **Capturing Phase** (Top → Down)
   2️⃣ **Target Phase** (Actual element)
   3️⃣ **Bubbling Phase** (Bottom → Up)

Visualizing Event Flow



Example DOM:

```
<body>
 <div>
  <button>Click</button>
 </div>
</body>
```

Clicking <button> triggers:

CAPTURE: body → div → button
TARGET: button
BUBBLE: button → div → body

🧠 2. Bubbling (Default Behavior)

By default, events **bubble upward**.

```
document.querySelector("div").addEventListener("click", () => {
 console.log("DIV clicked");
});

document.querySelector("button").addEventListener("click", () => {
 console.log("BUTTON clicked");
});
```

Click button → Output:

BUTTON clicked
DIV clicked

Because the event bubbled up.

3. Capturing (Less Used, But Powerful)

You can listen during the **capturing phase**:

```
div.addEventListener("click", handler, true);
```

true = capture mode.

Now it runs **before** the button handler.

4. Stopping Propagation

Sometimes you don't want bubbling.

```
button.addEventListener("click", (e) => {
  e.stopPropagation();
});
```

This prevents parent handlers from firing.

Used in:

- Modals
- Dropdowns
- Nested clickable components

How to add event listener

```
button.addEventListener("click", (event) => {
console.log(event.target); // actual clicked node
console.log(event.currentTarget); // where listener is attached
});
```

preventDefault() — Stop Browser Behavior

Stops built-in action like navigation or form submit.

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
});
```

Passive, Once, and Options (Advanced Optimization)

```
element.addEventListener("scroll", handler, {
  passive: true,
  once: true
});
```

Option	Purpose
passive	Improves scroll performance
once	Auto-remove listener
capture	Run in capture phase

Event Bubbling vs Event Delegation

Event Bubbling means when an event happens on a child element, it automatically **propagates upward to its ancestors**.

Example DOM

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
```

JS

```
document.getElementById("parent").addEventListener("click", () => {
  console.log("Parent clicked");
});

document.getElementById("child").addEventListener("click", () => {
  console.log("Button clicked");
});
```

When you click the button 👇

Output:

Button clicked
Parent clicked

The event fired on:

button → div → body → document

That upward movement is **bubbling**.

Event Delegation is a pattern where you attach **one event listener to a parent** instead of many listeners on children.

It relies on bubbling to detect which child triggered the event.

❌ Without Delegation (Bad for Performance)

```
document.querySelectorAll("li").forEach(li => {
  li.addEventListener("click", () => {
    console.log("Item clicked");
  });
});
```

If you have 1000 <li>:

➡️ You created 1000 listeners.

✅ With Delegation (Professional Way)

```
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    console.log("Item clicked:", e.target.textContent);
  }
});
```

Now:
✓ Only **ONE listener**

✓ Works for future elements
✓ Faster + scalable

Why Delegation Works

Because of bubbling:

User clicks
→ Event bubbles to
→ Parent handler inspects event.target
→ Decides what to do

You intercept the event **on its way up**.

In **event delegation**, the parent doesn't "know" beforehand which child will trigger — it simply listens for the bubbled event and then checks the **event.target**, which is the actual element that was clicked. Inside the parent handler, we use conditions like e.target.matches(".child") or e.target.closest(".child") to identify whether the event came from the specific child we care about, and run logic only for that element.

Q.Why does React use delegation internally?

React uses event delegation by attaching a single listener at the root and leveraging bubbling to handle events for all components, which reduces memory usage, avoids reattaching listeners during re-renders, and enables its cross-browser Synthetic Event system.

Performance & Real-World DOM Engineering

Q1: What Causes Forced Synchronous Layout?

Forced synchronous layout (also called **layout thrashing**) happens when JavaScript **reads layout information after modifying the DOM**, forcing the browser to **immediately recalculate styles + layout** before continuing.

You break the browser's optimization pipeline and demand:
"Give me layout NOW."

Browsers normally batch work like this:

JS → Style → Layout → Paint (async, optimized)

But forced layout changes it to:

JS → Layout NOW → JS → Layout AGAIN → JS → Layout AGAIN

✅ Q2: How to Measure DOM Cost Using Performance API?

You measure DOM + layout cost using the Performance API and DevTools timing.

DOM performance can be measured using the Performance API (performance.now, mark, measure) and profiling tools to track layout, style recalculation, and long tasks."